Metody analizy sieci złożonych

Zadanie nr 1 – zapoznanie się z podstawowymi narzędziami do analizy sieci złożonych oraz z podstawowymi miarami sieciowymi

0.1 *Neo4j*, *Python* i *NetworkX*

Neo4j (`https://neo4j.com`) to grafowa baza danych. Pozwala łatwo modelować złożone zależności dzięki obsłudze rónych typów węzłów oraz krawędzi. Umożliwia szybkie zapytania o relacje między danymi i jest często wykorzystywana w analizie sieci społecznościowych, rekomendacjach. Oficjalna dokumentacja. Inny bardziej zaawansowane przykłady: neo4j-gnn-llm-example, graph-machine-learning-examples.

NetworkX (`https://networkx.org/`) to pakiet do języka programowania Python, służący do tworzenia, manipulowania i badania struktury, dynamiki i funkcji sieci złożonych. Oficjalny przewodnik użytkownika znajduje się o tu. Inne przykłady: FirstCourseNetworkScience, jdfoote.

1 Dane

Studenci mogą pobrać dwie dowolne inne sieci z dowolnego repozytorium jako alternatywy to proponowanych danych w obu częściach zadania pod warunkiem, że będą miał podobny rozmiar, czyli dla sieci z części 1 700-1000 wierzchołków, a dla częsci 2 rzeczywistą sieć z 150+-15 wierzchołków. Przykładowe repozytoria zbierające dane o sieciach:

1. Konect `http://konect.cc/networks/`
2. Network Repository `https://networkrepository.com/`
3. Stanford Large Network Dataset Collection `http://snap.stanford.edu/data/`
4. The Colorado Index of Complex Networks `https://icon.colorado.edu/`

2 Zadanie 1, część 1

Część 1 zadania numer 1 realizowane jest indywidualnie w trakcie pierwszych zajęć. Średni czas realizacji zadania: 60 minut.

2.1 Dane

Z repozytorium Konect `http://konect.cc/networks/` pobierz następujący zbiór danych: A Song of Ice and Fire `http://konect.cc/networks/asoiaf/` - jest to sieć z serii powieści fantasy „Pieśń Lodu i Ognia” George'a R. R. Martina, gdzie każdy wierzchołek jest postacią, a krawędziami są połączone postacie wspominane razem w tekście. Każdy wiersz jest postaci: *<pierwsza postać, druga postać, ilość wspomnień.*

2.2 Polecenia

1. Zainstaluj neo4j desktop zgodnie z instrukcją,
2. Stwórz grafową baze danych (Create instance > wypełnij nazwę bazy oraz hasło),
3. Włacz pluginy: APOC oraz Graph Data Science,
4. Podłącz się do utworzonej bazy (Connect > Query),
5. Zapoznaj sie z interfejsem sekcji query,
6. Stwórz plik *asoiaf.csv* w oparciu o *out.asoiaf* gdzie usuniesz pierwszy wiersz, trzecią kolumnę oraz zamienisz spacje na przecinki,
7. Do folderu import przenieś plik zawierający sieć (Local instances > ... > Open > Instance folder),
8. Wykonaj cipher query do zaimportowania danych. W pierwszym kroku zdefiniuj ograniczenie unikalności dla węzłów reprezentujących postacie:

   ```
   CREATE CONSTRAINT character_id IF NOT EXISTS
   FOR (c:Character)
   REQUIRE c.id IS UNIQUE;
   ```

   Ograniczenie to zapewnia, że każdy węzeł o etykiecie `Character` posiada unikalną wartość właściwości `id`, co zapobiega tworzeniu duplikatów i usprawnia działanie operacji scalania danych.

   Następnie wczytaj dane z pliku `asoiaf.csv`:

   ```
   LOAD CSV FROM 'file:///asoiaf.csv' AS row
   WITH toInteger(row[0]) AS src, toInteger(row[1]) AS dst
   ```

   Każdy wiersz pliku odpowiada pojedynczej krawędzi grafu i zawiera identyfikatory dwóch połączonych wierzchołków. Dane zostały jawnie rzutowane na typ liczbowy w celu zachowania spójności typów.

   W dalszej kolejności, przy użyciu instrukcji `MERGE`, stworzysz/odnajdziesz węzły odpowiadające postaciom:

```
MERGE (a:Character {id: src})
MERGE (b:Character {id: dst})
```

Instrukcja ta gwarantuje, że każdy wierzchołek sieci występuje w bazie dokładnie jeden raz.

Na końcu utwórz relację nieskierowaną typu `INTERACTS_WITH` pomiędzy parami węzłów:

```
MERGE (a)-[:INTERACTS_WITH]->(b)
MERGE (b)-[:INTERACTS_WITH]->(a);
```

9. Zwizualizuj graf w sekcji explore (upewnij się że w ustawieniach explore masz dostosowane limity do swojej sieci),
10. W zakładce Graph Data Science zaaplikuj następujące algorytmy:
    (a) Stopień węzła (degree centrality),
    (b) Pagerank,
11. Zaaplikuj styl dla pagerank oraz wyeksportuj sieć jako obraz (EXPORT > Export screenshot),
12. Znajdź wierzchołek majacy najwyższe wartosci zaaplikowanych tych centralnosci,
13. Wyeksportuj widok gdzie bedą tylko wierzchołki o stopniu węzła wyższym niz 12 (wykorzystaj slicer),

3 Dyskusja po części 1

Jak odebraliście czytelność i intuicyjność neo4j desktop, czy łatwo było wam odnaleźć potrzebne funkcje? Jak oceniacie responsywność aplikacji, szybkość wykonywania zapytań oraz operacji? Czy zmiana layoutu (hierarchical, forced-based, circular) zmieniała Wasze postrzeganie struktury sieci?

4 Zadanie 1, część 2

Częśc 2 jest realizowane indywidualnie z terminem wysyłki rozwiązania do kolejnych zajęć. Średni czas realizacji zadania: 2 godziny. Celem zadania jest przejście od analizy wizualnej sieci (część 1) do analizy numerycznej (oba podejścia się uzupełniają).

4.1 Dane

Z repozytorium Konect `http://konect.cc/networks/` pobierz następujący zbiór danych: komunikacja mailowa przedsiębiorstwa produkcyjnego `http://konect.cc/networks/radoslaw_email/` - jest to lista zawierająca metadane dot. korespondencji e-mailowej, gdzie każdy wiersz jest postaci: *<id_nadawcy, id_odbiorcy, waga, unix_timestamp>* Waga wynosi zawsze jeden, można ją zignorować. Z Jeśli jakaś wiadomość od jednego nadawcy ma ten sam znacznik czasowy (timestamp) co kilka kolejnych rekordów, oznacza to, że wiadomość była wysłana do kilku odbiorców, przy czym zbiór danych nie zawiera informacji kto w wiadomości był TO/CC/BCC.

4.2 Polecenia

1. **Grafy losowe (model Erdős-Rényi)**
   (a) Wygeneruj sieć Erdős-Rényi o $N = 100$ i $p = 0.05$,
   (b) Wylistuj wszystkie wierzchołki i krawędzie,
   (c) Oblicz stopień (degree) każdego węzła a następnie stwórz histogram stopni węzłów,
   (d) Przygotuj dataframe oraz jego opis statystyczny z wykorzystaniem funkcji *describe*. DataFrame powinien zawierać metryki obliczone dla wygenerowanej sieci losowej, przy czym:
      - każdy wiersz odpowiada pojedynczemu wierzchołkowi grafu,
      - indeksem ramki danych jest numer wierzchołka grafu.

      Należy uwzględnić następujące metryki:
      i. Przewodnictwo (betweenness centrality),
      ii. Bliskość (closeness centrality),
      iii. Współczynnik grupowania (clustering coefficient),
      iv. Pagerank,
      v. Średnia najkrótsze ścieżka sieci (average shorthest path),
      vi. Średnicę sieci (diameter),
      vii. Liczbę komponentów (number of connected components),
      viii. Gęstość sieci (network density).
   (e) Ile jest komponentów (connected components) w grafie,
   (f) Zwizualizuj graf w taki sposób, aby rozmiar węzłów odpowiadał mierze PageRank,
2. **Sieci bezskalowe (model Barabási-Albert)**
   (a) Wygeneruj graf wedle modelu Barabási-Albert z $N = 1000$ i $m_0 = m = 3$
   (b) Zwizualizuj graf layoutem Kamada-Kawai

(c) Znajdź najbardziej centralny węzeł według miary pośrednictwa (betweenness), jaki ma numer?

(d) Jaka jest średnica grafu?

(e) Jakie różnice widzisz pomiędzy grafem Barabási-Albert i Erdős-Rényi.

3. **Praca z rzeczywistymi danymi**

(a) Zaimportuj zbiór *out.radoslaw_email_email* i zachowaj tylko pierwsze dwie kolumny (dodatkowo przeskocz dwa pierwsze wiersze), następnie stwórz z zaimportowanych danych graf nieskierowany.

(b) Sprawdź ile wierzchołków i krawędzi ma Twój graf a następnie pozbądź się wielokrotnych krawędzi i pętli. O ile spadła liczba krawędzi?

(c) Jaki jest stopień każdego węzła? Stwórz histogram stopni węzłów. Podobnie stwórz histogram dla miary pośrednictwa (betweenness) oraz bliskości (closeness). Zweryfikuj czy wierzchołki, które mają największe wartości tych miar, są równie wysoko we wszystkich rankingach (możesz użyć np. miary Kendalla).

(d) Ile jest komponentów (connected components) znajduje się w grafie? Czy przy takiej reprezentacji danych mogą być wierzchołki bez krawędzi?

(e) Jaka jest średnica grafu i czy jest dużo inna od średniej długości ścieżki w grafie?

(f) Stwórz graf na nowo, ale tym razem jako skierowany. Ustaw wagi krawędzi w grafie w taki sposób, aby waga pomiędzy wierzchołkiem v_1 a v_2 była wyrażona jako liczba wiadomości wysłanych przez v_1 do v_2 w stosunku do wszystkich wiadomości wysłanych przez węzeł v_1.

(g) Sprawdź czy w takim grafie skierowanym można dostać się z każdego wierzchołka do każdego innego.

5 Dyskusja po częsci 2

Jak odebraliście czytelność i intuicyjność API networkX, czy łatwo było wam odnaleźć potrzebne funkcje? Czy różnice były intuicyjnie widoczne na wizualizacji pomiędzy sieciami ER oraz BA? Czy zastosowanie współczynnika Kendalla potwierdziło korelację rankingów? Jaki wpływ ma kierunkowość na średnie długości ścieżek?

6 Raport

Do prowadzącego przesyłamy raport zawierający kod, odpowiedzi na pytania, komentarze, wizualizacje i wykresy najlepiej w formie notatnika (np. *Jupyter Notebook*) ale może też być w formie skryptu z odpowiednimi komentarzami lub w formie PDFa.

7 Literatura

[Jankowski2018] Jankowski J., Szymanski B., Kazienko P., Michalski R., Bródka P. (2018) Probing Limits of Information Spread with Sequential Seeding. Scientific Reports, 8(1), 13996,

[Kempe2003] Kempe, D., Kleinberg, J., & Tardos, É. (2003). Maximizing the spread of influence through a social network. In ACM SIGKDD pp. 137-146,

[NetworkX] Aric A. Hagberg, Daniel A. Schult and Pieter J. Swart, "Exploring network structure, dynamics, and function using NetworkX", in Proceedings of the 7th Python in Science Conference (SciPy2008), Gäel Varoquaux, Travis Vaught, and Jarrod Millman (Eds), (Pasadena, CA USA), pp. 11–15, Aug 2008